혼합 내쉬 균형 탐색과 최적화

주제 선정 이유

우리 학교는 자습 신청 시스템을 가지고 있다. 자습을 진행할 위치를 온라인 시스템을 통해 신청한 뒤 해당 위치로 이동하여 신청한 교시 동안 자습이 진행되게 된다. 학생 각자 개인적인 자습 위치에 대한 선호도가 나뉘지만, 특정한 좌석의 경우 여러 이유에 의해 많은 사람들이 원하는 경우가 있다. 하지만 신청할 수 있는 좌석의 개수는 정해져 있고, 먼저 신청한 사람이 그 자리를 선점하게 되기에 모든 사람이 자신이 만족하는 자리에 앉을 수 없게 된다. 이와 같은 현상을 게임이론을 접목하여 분석해보려고 한다.

이론적 배경

내쉬 균형이란 게임에 참가하는 다수의 플레이어가 모두 자신의 전략을 선택할 때, 아무도 자신의 전략을 스스로 변경하여 더 나은 결과를 얻을 수 없는 상태를 말한다. (Wikipidia)

플레이어의 전략은 정규형 게임 표현을 통해 가능하다. 정규형 게임은 플레이어들의 전략과 각 전략 조합에 따른 보상(보수, 효용)을 행렬이나 함수로 나타낸 게임 형태를 말한다.

이때 플레이어가 2 명일 경우 전략 집합의 형태로 표현된 전략의 조합에 따른 보수를 보수행렬을 통해 표현하는 것이다.

만약 플레이어가 3 명 이상일 경우에는 단순 행렬로는 전략에 따른 보수를 표현하기 어렵다. 따라서 행렬을 확장한 텐서를 사용하여 표현하게 된다. 텐서는 1 차원인 벡터나, 2 차원인 행렬과 다르게 3 차원 이상의 데이터를 저장하는 데에 사용된다. 플레이어가 3 명 이상이면 3 차원 이상의 구조가 있어야만 보수가 표현이 가능하기 때문에 텐서가 필요한 것이다.

전략은 하나의 전략만 선택하는 순수전략과 모든 전략을 확률적으로 섞어 사용하는 혼합전략으로 나뉜다.

정규형 게임에서 항상 내쉬 균형을 가지는 것은 아니다. 순수 전략을 사용할 경우 내쉬 균형을 가질수도, 가지지 않을 수도 있다. 가위바위보처럼 대칭적 구조를 가지는 게임의 경우 순수 전략을 사용할 경우에는 어떠한 전략을 선택해도 보수가 0 이 되어 내쉬 균형을 가지지 못한다. 반면에 혼합 전략에서 내쉬 균형은 모든 유한 게임에 대해 항상 존재한다는 것이 존 내쉬에 의해 증명되었다 (Nash, 1950).

탐구 내용 및 결과

좌석 배치 시뮬레이션을 위해서는 학생에 해당하는 게임에 참가하는 플레이어의 착석 전략을 구성해야 한다. 이때 모든 학생이 매일 같은 자리를 이용하지 않기 때문에 순수전략이 아닌 혼압전략을 사용해야 한다. 따라서 각 학생은 각 좌석에 앉을 확률을 가지게 될 것이다. 또한 각 학생은 다양한 이유로 특성 좌석에 대한 선호도를 가지고 있다. 따라서 이를 반영하기 위해 각 학생의 해당 좌석에 대한 선호도를 효용 계산에 반영해야 한다. 마지막으로 해당 좌석 자체에 대한 선호도와 무관하게 해당 좌석이 인기있는 정도와 주변에 얼마나 많은 사람들이 앉으려 하는지, 주변에 얼마나 많은 사람들이 있는지가 좌석 선택에 영향을 줄 수 있다. 따라서 각 좌석에 대해 해당 좌석과 주변 좌석을 고려한 밀집도도계산하고, 각 학생의 밀집도 기피 상수를 곱하여 이를 시뮬레이션에 반영할 것이다. 이때 밀집도 기피 상수가 음수가 될 수 있게 하여 오히려 다른 사람들이 많이 앉는 자리 주변에 함께 앉는 것을 선호하는 학생도 고려할 수 있도록 할 것이다.

1. 시뮬레이션 모델링

자습 위치 선택 시뮬레이션은 다음과 같이 모델링 할 수 있다.

$\mathcal{N}$: 학생 수

$\mathcal{M}$: 좌석 수

λ: 밀집도기피 상수

선호도 행렬

$$\mathcal{S} \in \mathbb{N}^{\mathcal{N} \times \mathcal{M}}$$

혼합 전략 행렬

$$P \in \mathbb{N}^{N \times M}$$

$$0 \le P_{ij} \le 1, \forall i \in \{1, \cdots, N\}, \forall j \in \{1, \cdots, M\}$$

$$\sum_{j=1}^{M} P_{ij} = 1, \forall i \in \{1, \cdots, N\}$$

α_i : 학생 i의 밀집도 기피 상수

밀집도 계산

$$c_j = \sum_{i=1}^{N} P_{ij}$$

기대 효용 계산

$$U \in \mathbb{N}^{N \times M}$$

$$U_{ij} = S_{ij} - \alpha_i \times c_j$$

=> 플레이어 i가 좌석 j를 선택했을 때의 총 효용

2. 시뮬레이션 구현

1) 전체 텐서 방법

전략에 따른 보수를 텐서로 표현하기 위해서는 N 차원 텐서가 필요하다. 각 플레이어의 전략의 개수는 좌석의 개수와 같기 때문에 M 개의 전략을 가질 것이고, 따라서 각 차원에는 M 개의 요소가 필요할 것이다. 하지만 이것이 요구하는 공간 복잡도를 계산해보면 M^N개의 숫자를 저장해야 한다. 우리가 저장할 혼합 전략은 확률로써 표현되기 때문에 정수가 아닌 실수형 자료형을 사용해야 하기에 숫자 하나당 4byte 의 메모리를 사용하게 된다. 따라서 전체 메로리 사용량은 $4 \times M^N$byte 가 된다. N 에 우리 학교 학생 수 약 120 을 입력하고, 학교에 학생 수와 비슷한 개수의 좌석이 있다고 가정한 뒤 M 에도 120 을 입력하게 되면 요구되는 메모리는 $1.18 \times 10^{241} GB \approx 1.12 \times 10^{235} PB$에 해당하기 때문에 일반적인 컴퓨터로는 연산을 수행할 수 없을 것이다. 따라서 계산을 제한된 시간 내에 완료하기

위해서는 최적화를 통해 공간 복잡도를 줄여야 한다.

2) 좌석 단위 효용 분석

이 시뮬레이션을 통해 계산하고자 하는 것은 각 좌석의 점유율, 기대 보수, 전체 손실이기 때문에 **모든 경우를 직접 고려하는 것이 아닌 좌석 단위로 고려하여도 최종 결과에 큰 영향을 미치지 않을 것이다**. 이는 밀집도 함수의 분해 가능성, 경사하강법 사용 가능성, 효용 함수의 오목서을 통해 증명이 가능하다.

(1) 밀집도 함수의 분해 가능성

기존의 효용 함수의 경우 모든 플레이어의 전략을 동시에 고려하여 계산하기 때문에 모든 전략의 경우의 수를 텐서의 형태로 표현하여 고려해야 했다. 하지만 좌석 문제의 경우 밀집도를 통해 효용을 계산하게 되는데 한 좌석의 밀집도의 경우 멀리 떨어져 있는 좌석에는 영향을 받지 않기 때문에 해당 좌석과 주변 좌석의 밀집도만 고려하면 된다. 따라서 **내쉬 평형을 이루기 위해서 내쉬 평형으로부터 얼마나 떨어진 상태인지를 손실함수를 통해 나타내고, 이 값을 경사하강법을 이용하여 최소가 되도록 만들기만 한다면 내쉬 평형점을 근사할 수 있을 것이다**.

(2) 경사하강법 사용 가능성

경사하강법은 함수의 최솟값을 찾는 알고리즘 중 하나로, 함수의 값이 낮아지는 방향으로 독립변수들의 값을 변경하여 함수의 경사의 반대 방향으로 이동하며 함수값이 최소가 되는 지점에 도달하는 방식으로 작동한다.

기존 방식의 내쉬 균형 탐색법은 모든 경우의 수를 개별적으로 계산하기 때문에 높은 시간 복잡도를 보였는데, 현재 이 탐구에서 시뮬레이션하고 있는 게임의 경우 가위바위보와 같은 다른 게임들과 달리 **독립변수에 대해 종속변수가 연속적인 성질**을 띤다는 것이다.

$$U_{ij} = S_{ij} - \alpha_i \times c_j$$

효용 함수에서 c_j를 제외하고 나머지는 모두 상수에 해당한다. 플레이어별로 정해진 값이 존재하고,

이는 전략에 무관하다. 따라서 밀집도에 해당하는 c_j만 변수라고 할 수 있는데 c_j는 P_{ij} 의 값의 합에 의해 영향을 받기 때문에 전략에 영향을 받는다. 따라서 효용 함수는 P 에 대해 미분 가능하다. 이러한 성질 덕분에 전체 경우의 수를 텐서로 표현하여 계산해보지 않고 **경사하강법을 이용해 사이에 있는 경우의 수에 대한 계산을 생략**하고 변수를 조절하여도 내쉬 균형점을 찾을 수 있다.

따라서 좌석 단위로 각 학생의 전략, 보수, 선호도를 행렬의 형태로 표현하고, 이를 통해 내쉬 균형을 탐색하여도 유사한 결과를 얻을 수 있을 것이다. 또한 경사하강법을 이용하여 손실이 최소가 되는 지점을 찾는 방식으로 균형점을 찾는 방식으로 구현하고, gpu cuda 를 시용한 병렬처리를 적용해준다면 더욱 빠르게 연산을 완료할 수 있을 것이다. 이 시뮬레이션에서의 경우 **손실 함수의 최소를 탐색**하게 될 것이고, **독립변수는 학생들의 전략**이 될 것이다.

3) 코드 구현

아래 코드는 시뮬레이션을 pytorch 라는 python 기반 라이브러리를 활용하여 구현한 것이다.

```python
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt
import numpy as np
import matplotlib

# 글꼴 설정
plt.rcParams['font.family'] = 'Malgun Gothic'
matplotlib.rcParams['axes.unicode_minus'] = False

# 디바이스 설정
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

우선 시뮬레이션의 사용할 라이브러리를 초기화한다.

```python
# 변수
N = 120 # 사람 수
M = 140 # 좌석 수
lambda_congestion = (torch.rand(N, 1) * 2 - 0.5).to(device) # 밀집도 기피 상수 (-0.5 ~ 1.5)
steps = 1000 # 연산 반복 횟수
lr = 0.1 # 전략 업데이트 비율 (학습률)
temperature = 2.0 # 확률 분포 조절

# 좌석 선호도
torch.manual_seed(0)
S = (torch.randn(N, M) * 2).to(device)

# 랜덤한 좌석 거리 (임시)
def create_stable_random_distance(M, min_dist=0.5, max_dist=10.0):
    torch.manual_seed(42)

    dist_matrix = torch.rand(M, M) * (max_dist - min_dist) + min_dist

    # 거리의 대칭성 적용
    dist_matrix = (dist_matrix + dist_matrix.T) / 2

    # 대각선은 0으로 설정 (자신과의 거리는 0)
    dist_matrix.fill_diagonal_(1e-6) # 0으로 나눠서 생기는 오류 방지

    return dist_matrix.to(device)
```

이후 시뮬레이션에 영향을 미치는 변수들의 값을 초기화한다. 변수에는 사람 수 N, 좌석 수 M, 밀집도 기피 상수, 연산 반복 횟수, 전략 업데이트 비율, 좌석 선호도가 있다. 확률 분포의 경우 초기 전략을 랜덤하게 설정하는 과정에서 이 랜덤한 값들의 분포를 조절하는 변수이다. Temperature 의 값이 높을수록 값들이 더 부드러운 값을 가지게 된다. 좌석 선호도의 경우 행렬을 이용하여 표현된다. X 행 Y 열에 있는 값이 X 번째 자리와 Y 번째 자리 사이의 거리가 된다. Y 행 X 열에 있는 값은 X 행 Y 열에 있는 값과 같아야 하기 때문에 거리의 대칭성을 적용하는 것이고, 자신과의 거리는 0 이기 때문에 행렬의 대각선 부분은 모두 0 으로 설정하는 것이다.

```python
# 거리에 따른 가중치
def distWeight(dist, alpha=0.3, normalize=True):
    weights = torch.exp(-alpha * dist) # 지수 감소 함수 사용

    if normalize:
        # 각 좌석별로 가중치 합을 1로 정규화 => 과도하게 큰 가중치 방지
        row_sums = weights.sum(dim=1, keepdim=True)
        weights = weights / (row_sums + 1e-8) # 0으로 나눠서 생기는 오류 방지

    return weights

# 거리 행렬 생성
dist = create_stable_random_distance(M)
weight = distWeight(dist)
```

이것은 좌석의 거리에 따른 가중치를 부여하여 혼잡도를 계산할 때 가까운 좌석이 우선시하여 고려되도록 한다.

```python
logits = torch.nn.Parameter(torch.randn(N, M).to(device))
optimizer = torch.optim.Adam([logits], lr=lr)

congestion_weight = 0.2 # weighted_congestion의 가중치

for step in range(steps):
    optimizer.zero_grad()

    # 전략 분포
    probs = F.softmax(logits/temperature, dim=1) # (N, M)

    # 밀집도 계산
    simple_congestion = probs.sum(dim=0) # (M,)

    # weighted_congestion 계산
    weighted_congestion = weight @ simple_congestion # (M,)

    # simple과 weighted을 1 : congestion_weight 로 가중평균
    combined_congestion = (1 - congestion_weight) * simple_congestion + \
                          congestion_weight * weighted_congestion
    congestion = combined_congestion.unsqueeze(0) # (1, M)

    # congestion 값을 제한
    congestion = torch.clamp(congestion, max=10.0) # 최대값 제한

    # 효용 계산
    utility = S - lambda_congestion * congestion

    # 전체 평균 기대 효용 계산
    expected_util = (probs * utility).sum(dim=1).mean()

    # loss 최소화 -> 기대 효용 최대화
    loss = -expected_util
    loss.backward()
    optimizer.step()

    if step % 20 == 0:
        mean_util_seat_68 = utility[:, 68].mean().item()
```

이것이 주된 시뮬레이션 코드이다. 계산한 기대효용에 -1 을 곱한 손실을 미분하여 구한 gradient 를 기반으로 optimizer 라는 내부 시스템이 전략의 분포를 업데이트한다. 이때 lr 의 값에 따라 optimizer 가 전략의 분포를 업데이트하는 비율이 달라지게 된다. lr 의 값이 높아지게 되면 균형점을 더 빨리 찾을 수 있지만, 과도하게 높은 값은 오히려 균형점 탐색에 방해되거나 잘못된 균형점을 찾을 수 있다.

나머지 코드의 경우 계산을 완료한 데이터를 기반으로 결과를 해석하고 이를 시각화하는 코드이다.

소스코드: https://gist.github.com/airhood/5f4ff78eafca3568b693fa2d1a605928

제언 및 소감

이번 탐구 과정에서 내쉬의 게임 이론에 대해 학습한 후에 이를 직접 구현하고 최적화하는 과정에서 조절하는 변수의 범위를 확실하게 지정하고, 계산을 통해 얻으려고 하는 결과가 무엇인지 명확하게 지정해야 한다는 것을 알게 되었다. 특히 시뮬레이션의 확실한 목적을 정하는 것을 통해 최적화 과정에서 계산에서 필요한 값과 필요하지 않은 값이 무엇인지 확실하게 구분할 수 있었고, 이를 통해 필요한 값만 근사하여 적은 시간 복잡도로 구하기 위한 방법을 생각해낼 수 있었던 것 같다.

또한 어떠한 **새로운 방법을 찾는 과정에서 해당 방법 또는 알고리즘이 이전 방법 대비 가지는 장점이 무엇이고, 그것을 달성하기 위해서 포기한 요소나 희생한 요소가 무엇인지, 희생한 요소가 있다면 그것이 결과에 미칠 영향이 무엇인지 분석해보는 것이 매우 중요함**을 느끼게 되었다.

이를 통해 탐구 외에 다른 수학 문제를 해결하는 과정에서도 **하나의 풀이만 사용하는 것이 아닌, 이를 대체할 수 있는 더 나은 풀이를 다양하게 고민**하고, 새로운 풀이를 찾았을 때 **다양한 풀이를 비교할 수 있는 능력이 필요**하다는 것을 알게 되었다.

참고 자료

https://en.wikipedia.org/wiki/Normal-form_game

https://ko.wikipedia.org/wiki/%EB%82%B4%EC%8B%9C_%EA%B7%A0%ED%98%95

Martin J. Osborne. (2000) An Introduction to Game Theory

Osborne, M. J. (2000). Selected chapters from draft of An introduction to game theory. University of Toronto. http://www.economics.utoronto.ca/osborne/igt/

Nash, J. F. (1950). Equilibrium points in *n*-person games. *Proceedings of the National Academy of Sciences*, *36*(1), 48–49.

https://chatgpt.com/share/6837fd50-c244-8010-829d-82afdb144d0b